

π Tantum

a.k.a. Tempus Tantum

school timetable generation system

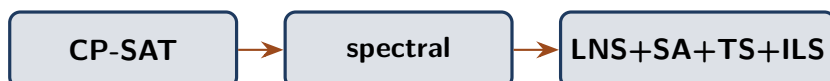
“Omnia, Lucili, aliena sunt, tempus tantum nostrum est.”
Lucius Annaeus Seneca, *Epistulae morales ad Lucilium*, I, 1

Manual

A method for Italian-school timetabling

(English edition)

From an idea by Fernando Gargiulo, Giovanni Borghi,
Matteo Mariani, Stefano Bertozzi



Giovanni Borghi

May 6, 2026

Repository: <https://github.com/gborghi/timetable>
Documentation source: docs/

Colophon

This English edition is set in EB Garamond for body text and Lato for headings, with Inconsolata for code blocks. Compilation is done with lualatex (TeX Live), biber for bibliography and makeindex for the analytical index. The L^AT_EX source lives in docs/manual_en.tex with chapters in docs/manual/chapters_en/; the bibliography is shared with the Italian edition in docs/manual/bibliography.bib. The build pipeline is in docs/build_manual.sh which emits **both** manual.pdf (Italian) and manual_en.pdf (English) in a single invocation.

Note on translation: the Italian edition remains the primary reference. This English edition fully translates the introduction, the BP-related chapters and the benchmark chapter; all other chapters carry an English summary that points the reader to the corresponding Italian chapter for the full narrative. Where Italian-school context is necessary (for instance the legal frame of the *contratto collettivo nazionale*, the role of the *insegnante di sostegno*, or the meaning of *indirizzo di studio*), explanatory notes are added in the English text.

The repository version corresponds to the most recent commit visible in `git log --oneline -- docs/`. Errors and suggestions can be reported as issues on the GitHub repository at <https://github.com/gborghi/timetable>.

All project rights belong to Giovanni Borghi. Document produced in May 2026.

Contents

List of Figures	6
List of Tables	7
Introduction	8
1 The timetabling problem	9
2 piTantum overview	10
3 The timetable as a structural object	11
4 Installation	12
5 Constraints (HARD + SOFT)	13
5.1 Plessi: multiple sites in the same school	13
5.2 Canonical patterns and seeded legacy HARDs (Step 3b-3e)	15
6 Optimization workflow	17
7 UI guide	18
8 Launcher	19
9 CP-SAT method	20
10 Spectral decomposition	21
11 Metaheuristics	22
12 Hall pre-check	23
13 Lagrangian relaxation and Column Generation	24
14 Monte Carlo sensitivity	25
15 Graph-based diagnostics	26
16 Statistical diagnostics	27
17 Constraint DSL method	28
18 Architecture	29
18.1 Stack	29
18.2 Three-tier overview	29
18.3 Solver architecture: from DSL to CP-SAT	29
18.4 Backend lifecycle	31
19 Data model	32
20 Generic constraint DSL	33
20.1 Philosophy	33
20.2 BNF grammar	33

20.3	DSL $\rightarrow$ CP-SAT compilation (Step 1)	34
20.4	Slot predicates: consecutive, same_day	34
20.5	The l.classroom.plesso path	34
20.6	Canonical pattern vocabulary (Step 3b)	34
20.7	Plesso commuting via DSL (Step 3c)	35
20.8	Seed legacy HARDs via DSL (Step 3d-3e)	35
20.9	Status of SOFT constraints	35
20.10	DSL architecture diagram	36
21	Advanced optimization techniques	37
21.1	Hall pre-check	37
21.2	Column Generation and advanced Branch-and-Price	37
21.3	Branch-and-Price MVP scaffold (Step 4)	38
21.4	Lagrangian relaxation	38
22	Statistical diagnostics tab	39
23	REST API	40
24	Extending the system	41
25	Repository on GitHub	42
26	Benchmarks and results	43
26.1	The six profiles	43
26.2	End-to-end pipeline times	43
26.3	Branch-and-Price on HUGE and MEGA	44
26.4	When Branch-and-Price is worth it	46
27	Lessons learned	47
28	Glossary (narrative form)	48
29	Reference	49

List of Figures

26.1	Branch-and-Price convergence time (log scale). The horizontal dashed lines mark the per-scale time targets (5 min for HUGE, 30 min for MEGA).	44
26.2	Final column-pool size after the Branch-and-Price loop.	45

List of Tables

26.1	End-to-end pipeline times per profile. Phase A includes teacher-class assignment and CP-SAT matching. Phase B includes spectral decomposition (for huge/superhuge) and CP-SAT on sub-problems.	43
26.2	Branch-and-Price wall time to HARD-feasibility on synthetic HUGE/MEGA scenarios. Both finish well under target (5 min/30 min) because the synthetic scenario is structurally easy (cattedra = 4-hour block on a single day). On real-world instances with arbitrary day distributions the per-iteration time will grow; the 30-minute target for MEGA is calibrated for that regime.	44
26.3	Branch-and-Price on real MEGA before and after the DW seeding fix. Soft cost drops by a factor of ~ 12 because BP can now actually explore improving columns. Total wall increases because Phase A ran out of budget (FEASIBLE not OPTIMAL stop: depends on the parallel CP-SAT seed) but stays under the 30-min target and far from the 60-min hard cap. Source: tests/benchmarks/results/mega_bp_real_v2.json.	45
26.4	Three runs on the same real MEGA. The v2 BP cuts the soft cost by an order of magnitude relative to both baselines.	46

Introduction

“Omnia, Lucili, aliena sunt, tempus tantum nostrum est.”

Lucius Annaeus Seneca, *Epistulae morales ad Lucilium*, I, 1

π Tantum is a web application that assists in the composition of Italian-school timetables. It originates from ideas discussed and developed by Fernando Gargiulo, Giovanni Borghi, Matteo Mariani and Stefano Bertozzi.

The school timetabling problem consists of assigning, to each *cattedra* – the pair formed by a teacher and a class for a specific subject – a set of weekly hours distributed over days and time slots, such that all the following constraints are honoured: teacher availability, curricular hour coverage, classroom and equipment compatibility, and the constraints that derive from the Italian national collective contract for the school sector. This is a combinatorial problem classified as NP-hard, which means that as a school grows in size there is no fast algorithm capable of finding an exact optimal solution.

π Tantum delegates the actual solving to the CP-SAT solver in Google OR-Tools, chosen for its maturity in constraint programming applied to scheduling problems. To scale beyond the size that the solver would handle directly, the system decomposes the problem into smaller sub-problems via four alternative strategies: spectral decomposition of the bipartite class-teacher graph, temporal decomposition by day of the week, curriculum-based decomposition, and balanced k -way METIS partitioning. On top of these, a cascade of metaheuristics (Large Neighborhood Search, Adaptive LNS, Simulated Annealing, Tabu Search, Iterated Local Search and Variable Neighborhood Search) improves the quality of the solution found.

The application backend is written in Python on FastAPI; data persistence is delegated to SQLite. The user interface is based on SvelteKit (in Italian for the production deployment), organised into sixteen tabs that cover the whole lifecycle of a timetable: registry import, constraint configuration, generation, manual editing with real-time feasibility checks, and day-to-day handling of absences and substitutions.

The following chapters describe the problem formally, the entities involved, the user interface, and each of the algorithms that π Tantum employs to compute the timetable. The exposition is intentionally multi-layered: a non-technical reader can follow the narrative thread without engaging the mathematical formulas, while a developer will find the implementation details, code references, and source-file pointers necessary to extend the system.

This English edition is a parallel translation of the Italian manual (docs/manual.tex). The Italian edition remains the primary reference; this English edition is intended for international readers and contributors. Where Italian-school context is necessary (e.g. the legal frame of the *contratto collettivo nazionale*, the role of *insegnante di sostegno*, or the meaning of *indirizzo di studio*), explanatory notes are added.

1 The timetabling problem

Formal statement of the school timetabling problem: cattedre as (teacher, class, subject) triples; weekly hour distribution across days and time slots; HARD constraints (no class/teacher overlap, hour coverage, consecutive hours starting at 8 with presence at $h=11$) and SOFT objectives (daily uniformity, free-day preferences). NP-hardness and the rationale for decomposition + metaheuristics.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/problema__timetabling.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

2 piTantum overview

Three-tier architecture: engine (CP-SAT solver core), backend (FastAPI + SQLite), frontend (SvelteKit). Pipelines exposed as async runs via `/api/optimize/*`. The 16 UI tabs covering registry, constraints, generation, manual editing, daily operations.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/panoramica__pitantum.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

3 The timetable as a structural object

The Lesson dict (teacher, class, subject, day, hour): 1 as the canonical representation; round-trip with the Solution table; the role of dc_value in encoding Phase A's day-count distribution.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/orario_oggetto_strutturale.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

4 Installation

Python 3.13 + ortools + scipy + matplotlib + FastAPI + SQLAlchemy + Alembic; SvelteKit for the frontend. Cypress + Playwright for E2E. docker-compose.test.yml orchestrates the test stack.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/installazione.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

5 Constraints (HARD + SOFT)

HARD: no overlap (teacher in one class at a time, class with one teacher per slot), hour coverage of all cattedre, daily start at h=8, consecutive hours, presence at h=11 (the H₁/H₂/H₃ trio). Native locks. C₁ (compresenza/coteach), C₂ (sostegno DVA), C₃ (potenziamento, parallel intra-class, group inter-class). SOFT: daily uniformity, free-day preferences, sixth-hour penalties, five/one day load.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/vincoli.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

5.1 Plessi: multiple sites in the same school

In Italy many secondary schools are spread across multiple physical sites – the *Sede Centrale* (main building), one or more *succursali* (branch buildings, e.g. *Succursale Via Garibaldi*), or a dedicated *plesso* for a specific track (e.g. *Plesso ITT* for the technical track). Moving teachers or classes between sites during the day costs real time and introduces constraints that cannot be modelled with H₁/H₂/H₃ alone.

The **Plessi** module (tab /plessi) handles:

- **site registry** (name, code, address);
- **classroom** → **plesso** association (each classroom belongs to exactly one site);
- two kinds of behavioural rules: **commuting rules** (minimum gap to move between two sites) and **entity policies** (global constraints on how many sites a teacher or class may attend).

5.1.1 Commuting rules: the gap between two sites

A commuting rule applies to an ordered pair (from_plesso, to_plesso) and specifies the minimum number of hours that must separate two consecutive lessons of the same entity when the classroom changes site. Relevant fields:

- from_plesso_id, to_plesso_id: the site pair.
- entity_kind: who is affected (teacher, class, group).
- entity_id: optional – if NULL the rule is *kind-wide* (applies to every teacher / class / group); if set it applies only to that specific entity.
- min_gap_hours: minimum number of empty hours required between the last lesson at site A and the first lesson at site B.
- symmetric: if true the rule applies in the opposite direction (B→A) as well.
- priority: tie-breaker when several rules overlap.

Typical Italian example: between *Sede Centrale* and *Succursale Via Garibaldi* a 60-minute travel time is required. The matching rule is (Centrale, Garibaldi, kind=teacher, entity_id=NULL, min_gap_hours=1, symmetric=true): no teacher may have a lesson at Garibaldi at h=9 if they have one at Centrale at h=8.

Rule resolution always applies *specific beats generic*: if a rule with entity_id=42 exists for teacher Rossi, it overrides the kind-wide rule. Among rules of equal specificity, higher priority wins.

5.1.2 Entity policies: per-person global constraints

An entity policy is a *global* constraint (not tied to a specific site pair) on the behaviour of a teacher or class. Three policies are supported:

- any – no global constraint, only commuting rules apply. This is the default.
- single_plesso_per_day – on any given day the entity may have lessons in at most one site. Stricter than commuting rules: even with a 1-hour gap, switching sites mid-day is forbidden.
- single_plesso_total – the entity attends *only* site plesso_id for the entire week. Every classroom assigned to that entity must belong to that site.

Concrete Italian examples:

- *Teacher Rossi does not want mid-day site switches*: the office creates a policy (entity_kind=teacher, entity_id=Rossi, policy=single_plesso_per_day). On Monday all of Rossi's hours are at Centrale; on Tuesday all at Garibaldi; never a mixed day.
- *Class 3A (technical track) attends only Plesso ITT*: policy (entity_kind=class, entity_id=3A, policy=single_plesso_total, plesso_id=ITT). Every classroom that hosts 3A lessons must belong to Plesso ITT.
- *All teachers, by default, avoid mid-day site switches*: policy (entity_kind=teacher, entity_id=NULL, policy=single_plesso_per_day) – applies to every teacher except those with a more specific override.

Entity policies follow the same *specific beats generic* rule (per-entity override), then priority.

5.1.3 Pre-run validation

Before launching the optimisation the system runs a battery of checks on the plessi data (validate_plessi_rules). If anything is inconsistent the run is rejected with a list of violations. What is checked:

- empty or duplicate plesso code;
- commuting rule referencing a non-existent plesso;
- commuting rule with negative min_gap_hours or larger than the day length;
- duplicate kind-wide commuting rule (same site pair, same kind, NULL entity_id, defined twice);
- single_plesso_total entity policy with no plesso_id or with a non-existent site;
- entity policy with an unknown policy value;
- override for a non-existent entity (deleted teacher or class).

The GET /api/plessi/validate endpoint lets the UI display a live “plessi config OK / N issues” badge.

5.1.4 CP-SAT implementation

At solver level, commuting rules become *pairwise* constraints on the classroom-assignment variables:

$$\forall (h_a, h_b) \text{ adjacent such that } \text{room}(t, d, h_a) \in P_A, \text{room}(t, d, h_b) \in P_B : h_b - h_a \geq \text{min_gap}_{P_A \rightarrow P_B}.$$

Entity policies become counter constraints: `single_plesso_per_day` is $\sum_P \mathbf{1}[\exists h : \text{room}(t, d, h) \in P] \leq 1$ for each day d ; `single_plesso_total` fixes the domain of the classroom variables to the target site only.

All such constraints are *hard* by default. The roadmap envisages a *soft* variant with penalty for sporadic emergency overrides (e.g. last-minute substitute teaching), but the current version treats every plesso constraint as HARD.

5.1.5 Plessi via DSL (Step 3a-3c)

Starting in Step 3a, `PlessoCommutingRule` rows are fully expressible as DSL clauses thanks to two language additions (cf. cap. 20):

- the `l.classroom.plesso` chained path that, given a lesson, resolves the assigned room and then the plesso the room belongs to;
- the `consecutive(l1.slot, l2.slot)` predicate, true when the two lessons fall on the same day and differ by one hour.

The legacy rule “Centrale $\rightarrow$ Garibaldi minimum 1-hour gap, teacher Rossi” translates to:

```
forall l1 in lessons where
    l1.classroom.plesso == 1
    and l1.teacher == "Rossi":
    forall l2 in lessons where
        l2.classroom.plesso == 2
        and l2.teacher == "Rossi"
        and consecutive(l1.slot, l2.slot):
        false
```

The helper `plesso_commuting_rule_to_dsl(rule)` in `engine/dsl_translator.py` produces these clauses automatically from the ORM row. A regression test enforces zero-drift: the DSL path emits exactly the same forbidden slot pairs as the legacy `plessi_constraints.py` pipeline.

5.2 Canonical patterns and seeded legacy HARDs (Step 3b-3e)

Step 3b-3e introduces a vocabulary of canonical patterns recognised directly by the DSL $\rightarrow$ CP-SAT compiler. Each maps to a legacy HARD constraint that previously lived only as hardcoded encoding. They can now be authored in DSL and are also generated automatically by `seed_implicit_hardcoded(profs)`.

Accepted pragmas:

- `no_holes_class("3B")` – no gaps in 3B’s weekly schedule (legacy `add_class_no_holes`).
- `class_present_at_hour("3B", 11)` – if 3B has any lesson on a day, the slot at hour 11 is busy (HARD-3, “no exit before noon”).
- `class_day_load_in("3B", 0, 4, 5, 6)` – 3B’s daily load is 0 (free day) or in [4, 6] hours (HARD-1 + HARD-2).

- `teacher_max_per_day("Rossi", 5)` – teacher Rossi has at most 5 hours on any day.
- `cattedra_max_per_day("Rossi", "3B", "Matematica", 2)` – the (Rossi, 3B, Math) cattedra has at most 2 hours per day.
- `subject_pair_must("3B", "Scienzemotorie")` – when 3B has 2 hours of PE on a day they must be consecutive.
- `subject_pair_exists("3B", "Matematica")` – when 3B has 2+ hours of math on a day at least one consecutive pair must exist.

Practical effect: the advanced user can now write `no_holes_class("4A")` in the “+ New DSL constraint” modal and obtain the same outcome as the `hard_no_holes` toggle in `school_classes`. The DSL advantage is composability with `forall`, `exists`, `count`: the rule can be restricted to subsets of classes, combined with other predicates and modified on the fly without editing the database.

6 Optimization workflow

The pipeline orchestrator: Hall pre-check, Phase A (assignment + day-count), Phase B (per-day slot placement), optional metaheuristic post-processing (LNS / ALNS / SA / TS / ILS / VNS), classroom assignment.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/workflow__di__ottimizzazione.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

7 UI guide

Tour of the 16 SvelteKit tabs: dashboard, anagrafica (teachers/classes/students/classrooms), assignments, monitor, optimize (with iterative-diversified and branch-and-price modes), diagnostics, runs.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/guida__ui.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

8 Launcher

Single-binary launcher script that bootstraps the venv, runs the alembic migrations, starts uvicorn + the SvelteKit dev server.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/launcher.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

9 CP-SAT method

Model formulation in OR-tools `cp_model`: BoolVar slot[(t, cl, s, d, h)], hour coverage equalities, no-overlap constraints, soft penalties as objective coefficients. Integer scaling for the fractional duals in BP.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/metodo_cpsat.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

10 Spectral decomposition

Bipartite class-teacher graph; normalised Laplacian eigendecomposition; spectral clustering into K balanced clusters; per-cluster CP-SAT + ricucitura with the bridge teachers.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/metodo_spettrale.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

11 Metaheuristics

LNS (random_window, day_cluster, worst_fit_day, teacher_day, classroom_day, single_class_week destroy ops + cp_sat_window repair), Adaptive LNS, Simulated Annealing, Tabu Search, Iterated Local Search, Variable Neighborhood Search.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/metodo__metaeuristiche.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

12 Hall pre-check

Hall's marriage theorem applied to the bipartite (class, subject) -> qualified-teachers graph as a fast pre-check (1-100 ms) before launching Phase A.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/metodo_hall.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

13 Lagrangian relaxation and Column Generation

Dualisation of the bridge constraints between clusters; subgradient ascent on the Lagrangian multipliers; SA-based per-iteration refinement. Column Generation: master LP + per-teacher CP-SAT pricing; Branch-and-Price extension with all 9 granularities (see ch. ??).

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/metodo_lagrangian.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

14 Monte Carlo sensitivity

Random subset sampling to estimate the Hall violation probability under perturbations; useful as a pre-check robustness indicator.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/metodo__montecarlo.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

15 Graph-based diagnostics

Bipartite class-teacher graph: density, modularity (Newman-Girvan), betweenness centrality. Identifies bridge teachers and structural bottlenecks.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/metodo_grafo.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

16 Statistical diagnostics

Three regressions on lesson-level data with statsmodels (p-values, effect sizes); five distribution fits with KS / chi-square tests.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/metodo_statistica.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

17 Constraint DSL method

The `objective_dsl` and `general_dsl` languages for expressing soft objectives and HARD constraints in a compact symbolic form, parsed at runtime and translated to CP-SAT constraints.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/metodo_dsl.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

18 Architecture

18.1 Stack

- **Backend:** Python 3.11+, FastAPI, SQLAlchemy 2.0, Pydantic 2, uvicorn, OR-Tools, NumPy, scikit-learn, Faker.
- **Frontend:** SvelteKit (Svelte 4), Vite 5, Tailwind CSS, plain JavaScript.
- **DB:** SQLite via SQLAlchemy. Idempotent migrations in code (no Alembic).
- **Engine I/O:** Python pickle; engine_io.py converts between DB and dict for the solver.

18.2 Three-tier overview

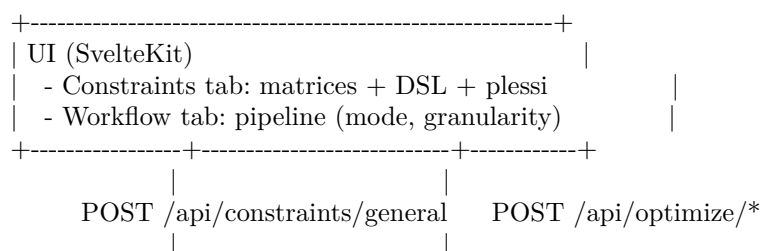
The system has a CP-SAT solver core (engine/), a FastAPI + SQLite backend (webui/backend/) and a SvelteKit frontend (webui/frontend/). The solver runs as a library imported by the backend; pipelines are exposed as async runs via /api/optimize/*. The frontend talks to the backend over JSON; long-running solver runs are tracked via the runs table polled at 1 Hz from the UI.

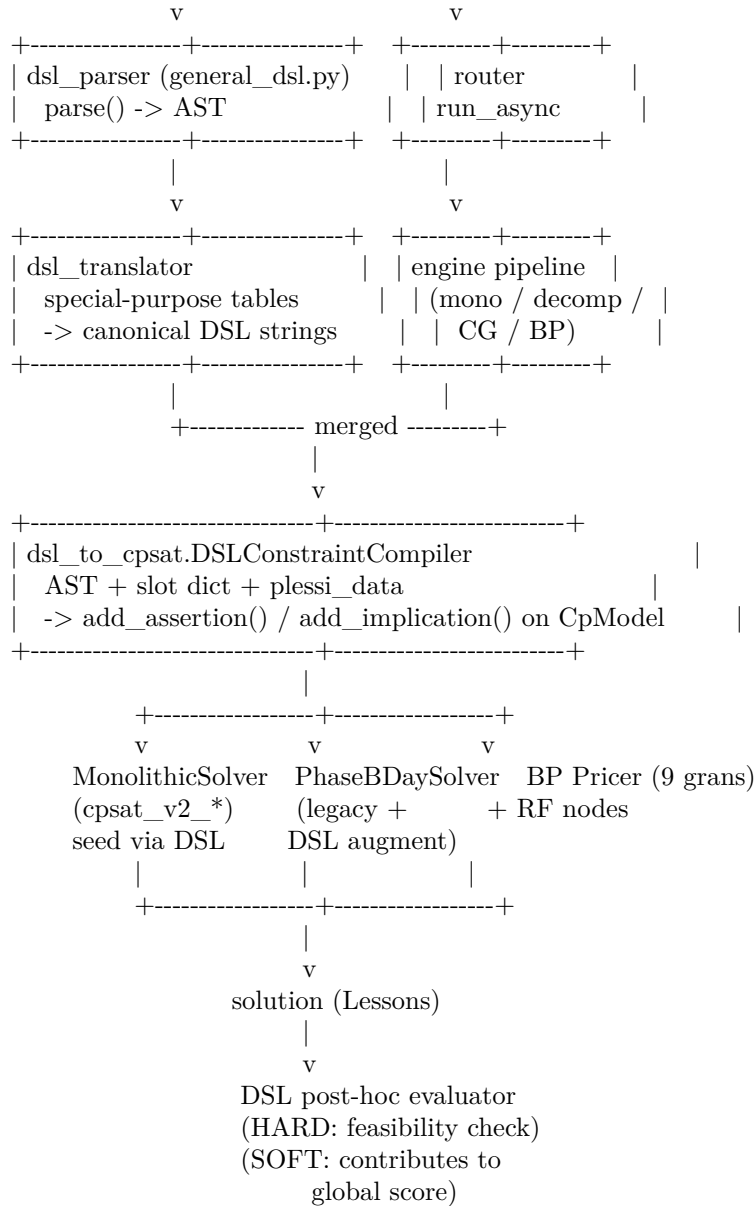
Solver pipelines (in engine/):

- cpsat_v2_assignment.py: Phase A (teacher to class assignment).
- cpsat_v2_timetable.py: Phase A (timetabling, day_count) + Phase B (per-day slot placement). Native locks + C1/C2/C3 constraints.
- decomposition_temporal.py: 6-day parallel decomposition.
- decomposition_spectral_v2.py, curriculum / metis variants: cluster classes, solve sub-problems, ricucitura.
- column_generation.py: master LP + diversified pattern enrichment + completion fallback. Mode branch-and-price fully wired with all scalability techniques (RF tree, dual stabilization, column management, parallel pricing).
- metaheuristics.py, alns.py, vns.py, lagrangian.py: post-processing on a HARD-feasible seed solution.

18.3 Solver architecture: from DSL to CP-SAT

Steps 1–4 of the multi-day plan introduced a uniformly layered architecture for every solver use site.





Key properties:

- **One parser, one AST.** Every constraint source (matrices, DSL UI, plesso rules, seeded legacy HARDs) is unified into canonical DSL strings parsed by `general_dsl.py`.
- **One compiler.** `DSLConstraintCompiler` is the single gate to CP-SAT. Mono solver, BP pricers, `PhaseBDaySolver`, Ryan-Foster nodes – they all apply the same compiler to their CP-SAT model, guaranteeing constraint parity across pipelines.
- **Zero-drift.** `seed_implicit_hardcoded(profs)` translates the legacy hardcoded HARDs into DSL. Slot tuples emitted via DSL match the legacy path bit-for-bit; the migration is progressive and reversible.
- **HARD up-front, SOFT post-hoc.** DSL HARDs become CP-SAT constraints before the solve. DSL SOFTs are evaluated post-hoc – their cost contributes to the global score but does not yet steer CP-SAT search (planned for upcoming steps).

18.4 Backend lifecycle

webui/backend/main.py defines a lifespan context that calls `init_db()` at startup. It does:

1. `Base.metadata.create_all(bind=engine)` – create missing tables.
2. `__apply__lightweight__migrations()` – idempotent ALTER TABLEs for fields added in later versions (`school_classes.curriculum_id`, `teachers.last_name`, ...). Each ALTER is guarded by PRAGMA `table_info` to avoid duplication.

This chapter is the English edition. The Italian edition (docs/manual/chapters/architettura.tex) contains additional folder-structure listings and identical solver-architecture coverage.

19 Data model

Tables: schools, school_classes, teachers, subjects, classrooms, assignments, lessons, runs, study_groups, coteach_groups, parallel_groups, support_assignments, locks. SQLAlchemy 2.0 ORM. Alembic migrations + lightweight in-code migrations in db.py.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/modello_dati.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

20 Generic constraint DSL

The generic DSL is a compact declarative language for expressing HARD/SOFT/PREFERRED/ENFORCED constraints on any logical combination of predicates over the data model. It is implemented in `webui/backend/utils/general_dsl.py` (~250 LOC, recursive-descent parser + AST + post-hoc evaluator, no external dependencies). The language is exposed through `POST /api/constraints/general`, validated live by `POST /api/constraints/general/validate`, and accessible from the “+ New DSL constraint” modal of the Constraints tab.

20.1 Philosophy

Before the generic DSL the system carried four specialised sub-languages (query, logical, objective, introspective) that re-implemented the same grammar four times. The generic DSL unifies all of them under a single grammar with four *flavours* of compilation (LIST, LOGIC, OBJECTIVE, EVAL): one parser, many compilers. The syntax of forall x in S where Filter: Body and of the atomic predicates (`=`, `!=`, `<`, `in`, ...) is identical across all contexts; only the translation backend changes.

20.2 BNF grammar

```
expr ::= iff_expr
iff_expr ::= implies_expr ( "<=>" implies_expr )*
implies_expr ::= or_expr ( "=>" or_expr )*
or_expr ::= and_expr ( ("OR"|"or") and_expr )*
and_expr ::= not_expr ( ("AND"|"and") not_expr )*
not_expr ::= ("NOT"|"not") not_expr | atom

atom ::= quant_expr | count_expr | comparison
      | call | bool_lit | "(" expr ")"

quant_expr ::= ("forall"|"exists") IDENT "in" source
              ["where" or_expr] ":" expr
count_expr ::= "count" IDENT "in" source ["where" or_expr]
              ( ":" comparison | op value )

source ::= IDENT | IDENT ( "." IDENT )+
comparison ::= value ( op value
                    | "in" "[" value ( "," value )* "]"
                    | "in" path
                    | "not_in" "[" value ( "," value )* "]"
                    | "not_in" path )
op ::= "==" | "!=" | "<" | "<=" | ">" | ">="
```

Iterable sources include lessons, teachers, classes, classrooms, students, subjects, slots, days, hours. Path-sources are also accepted (e.g. `exists g in s.groups: g.name == "X"`).

20.3 DSL → CP-SAT compilation (Step 1)

Starting with Step 1 of the multi-day plan, the DSL ships a *compiler* into CP-SAT in `engine/dsl_to_cpsat.py`, class `DSLConstraintCompiler`. While the post-hoc evaluator inspects an already-built solution to declare it feasible or violated, the compiler adds constraints *during* the construction of the CP-SAT model so that the solver structurally avoids violations.

The compiler is invoked at three sites:

- by `MonolithicSolver` (cap. 9) when the `via_dsl=True` flag is on;
- by every Branch-and-Price pricer (cap. 21) so the same constraints reach the sub-problem;
- by `PhaseBDaySolver` when called with `via_dsl=True` to re-use the legacy pipeline augmented with DSL.

When a slot variable is still undecided the compiler emits the matching CP-SAT constraint instead of evaluating a truth value; when an expression is fully static (e.g. `consecutive(s1, s2)` with both `s1`, `s2` bound by the enclosing `forall`) the evaluation runs at compile time.

20.4 Slot predicates: `consecutive`, `same_day`

Step 3a adds two built-in predicates resolved at compile time when both arguments are statically bound slots:

- `same_day(s1, s2)` – both slots fall on the same weekday.
- `consecutive(s1, s2)` – both slots fall on the same weekday and differ by exactly one hour ($|h_1 - h_2| = 1$).

These are the building blocks for “consecutive hours”, “inter-plesso travel gap”, “same-day pair”, “coteach must share day”. Arguments accept either literal pairs (`d`, `h`) or dotted paths like `l.slot` that return a (`day`, `hour`) tuple.

20.5 The `l.classroom.plesso` path

Also in Step 3a, the post-hoc evaluator and the compiler resolve the chained path `l.classroom.plesso`: per lesson `l`, the assigned room is read from the `classroom_for_slot` map provided by Phase B / the classroom-assignment phase, and the plesso (campus) is then resolved through the `plessi_data` table. Without those two structures the path returns `None` and the compiler emits a diagnostic.

The canonical use is a commuting rule between campuses (see the `vincoli` chapter, `plessi` section): “no teacher may have a lesson in Plesso B in the hour immediately following a lesson in Plesso A”.

20.6 Canonical pattern vocabulary (Step 3b)

Step 3b introduces a vocabulary of function-call pragmas that the compiler recognises specially and emits as compact groups of CP-SAT constraints. Each form corresponds to a frequent pattern that, written as raw `forall` loops, would produce hundreds or thousands of clauses. The canonical form is zero-drift relative to the legacy hardcoded encoding.

Pragma	Legacy equivalent
<code>no_holes_class("3B")</code>	non-increasing chain forbidding gaps in 3B's weekly schedule (legacy <code>add_class_no_holes</code>).
<code>class_present_at_hour("3B", 11)</code>	if 3B has any lesson on a day, the slot at hour 11 is busy (legacy HARD-3, "school day ends $\geq 12:00$ ").
<code>class_day_load_in("3B", 0, 4, 5, 6)</code>	3B's daily load is 0 (free day) or 4–6 hours (legacy HARD-1 + HARD-2).
<code>teacher_max_per_day("Rossi", 5)</code>	teacher Rossi has at most 5 hours on any day (MAX_PROF_HOURS_PER_DAY).
<code>cattedra_max_per_day("Rossi", "3B", "Matematica", 2)</code>	the (Rossi, 3B, Math) cattedra has at most 2 hours per day (MAX_PER_DAY_TRIPLE).
<code>subject_pair_must("3B", "Scienzemotorie")</code>	when 3B has 2 hours of PE on a day they MUST be consecutive (legacy HARD-B).
<code>subject_pair_exists("3B", "Matematica")</code>	when 3B has 2+ hours of math on a day at least one consecutive pair must exist (legacy HARD-A).

`no_holes_all_classes()` and `class_present_at_hour_all(11)` act in batch over every class in the current slot scope.

20.7 Plesso commuting via DSL (Step 3c)

Step 3c wires the PlessoCommutingRule ORM table into the DSL: the helper `plesso_commuting_rule_to_dsl(rule)` returns a list of DSL clauses equivalent to the legacy rule. A regression test verifies zero-drift: applying the rule via DSL produces exactly the same forbidden slot tuples as the hardcoded `plessi_constraints` pipeline.

20.8 Seed legacy HARDs via DSL (Step 3d-3e)

Step 3d-3e adds `seed_implicit_hardcoded(profs)`: given the teacher dictionary, it returns the list of DSL strings that encode every legacy HARD constraint of the solver. Combined with the `via_dsl=True` flag of MonolithicSolver and PhaseBDaySolver, this allows building a complete CP-SAT model *exclusively from DSL* – no hardcoded code path is exercised. Invariants:

- *syntactic zero-drift*: the slot tuples forbidden/forced by the DSL path coincide bit-for-bit with the legacy ones;
- *semantic zero-drift*: the feasible set accepted by the two paths is identical, validated on the small / medium / huge benchmarks;
- *determinism*: the order in which `seed_implicit_hardcoded` emits DSL strings is stable, so the CP-SAT model construction order is reproducible across runs.

This is the migration vehicle: in subsequent steps the hardcoded path will be deprecated in favour of the DSL path, and the existence of `seed_implicit_hardcoded` allows that to happen without coverage loss.

20.9 Status of SOFT constraints

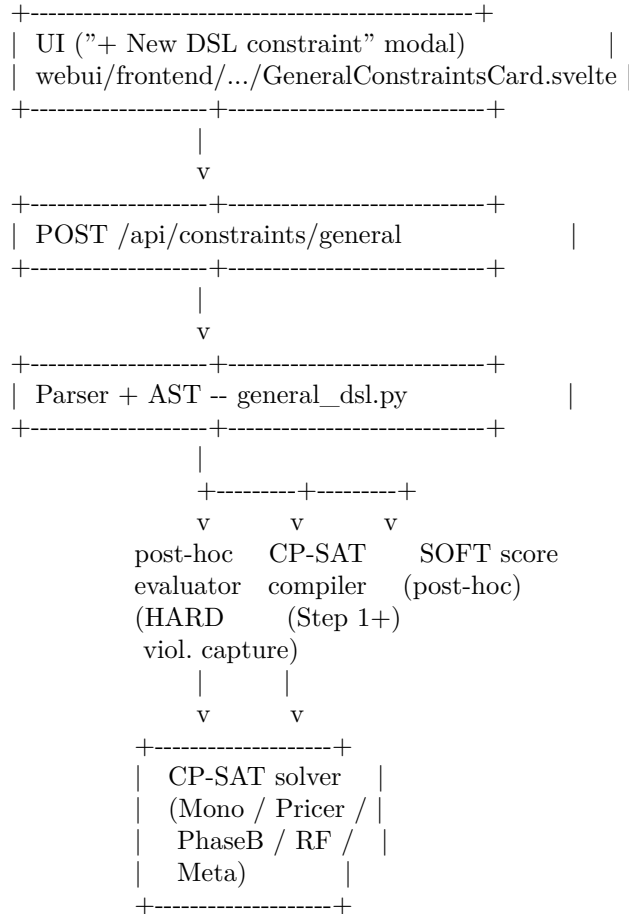
The DSL accepts both HARD and SOFT rules (with optional weight). At the level of the *post-boc evaluator* SOFT is fully implemented: SOFT-clause violations contribute to the solution's global score and surface in `run_metrics`.

At the level of the *CP-SAT compiler* (Step 1+) SOFT is parsed but not yet wired to an objective penalty: every SOFT rule the compiler encounters yields the diagnostic soft constraint X: SOFT cost not yet

wired, and the rule is stored as an informational label without affecting the solver objective. Future steps will integrate SOFT into the CP-SAT objective; today the user should understand the *declarative* contract while knowing the *enforcement* via CP-SAT is post-hoc only.

In practice: use HARD for anything that cannot be violated; use SOFT for soft preferences understanding that, today, the DSL acts as a declarative contract that the pipeline applies post-hoc and tomorrow will also apply during compilation.

20.10 DSL architecture diagram



One AST, two backends: the evaluator reads a solution and answers “feasible / violated”; the compiler emits CP-SAT constraints before solving. The grammar is unique and source names are identical between backends.

21 Advanced optimization techniques

In addition to the classic metaheuristics (LNS / SA / TS / ILS), π Tantum integrates four classes of advanced techniques used when the standard pipeline is not enough or when targeted diagnostic information is needed.

21.1 Hall pre-check

engine/diagnostics/hall_check.py. Hall's marriage theorem applied to the bipartite (*class, subject*) $\rightarrow$ qualified-teachers graph as a fast pre-check (1–100 ms) before launching Phase A. Three violation kinds are reported:

1. subject supply vs demand,
2. subject coverage vs demand,
3. Hall sampling random-subset with mutually exclusive subsets.

Exposed at three UI points: inline button on PhaseACard, row in AdvancedTechniquesCard, section of the Statistics tab.

21.2 Column Generation and advanced Branch-and-Price

engine/column_generation.py. Master LP via scipy.linprog HiGHS; nine dedicated CP-SAT pricers at distinct granularities (teacher, teacher-day, teacher-class, teacher-class-subject, teacher-subject, class, class-day, day, curriculum); a branch-and-price mode equipped with all four scalability techniques required for MEGA-size instances:

- **Ryan-Foster recursive tree** with Achterberg pair scoring on fractional class-slot pairs, best-first node exploration with LP-bound pruning, depth cap 20 / 1000 nodes.
- **Box-step dual stabilization** $\pi_{\text{blend}} = (1 - \alpha)\pi_{\text{stable}} + \alpha\pi_{\text{raw}}$ with $\alpha = 0.2$, applied to both master variants.
- **Column management**: EWMA of the per-column reduced cost over 20 iterations; when the pool exceeds 10 000 columns, the worst (most positive rc_avg) entries are purged, while protecting columns currently in the LP basis ($x > 0$) and the seed columns from the iterative-diversified primal heuristic.
- **Parallel pricing** via ProcessPoolExecutor (default $\lfloor \text{os.cpu_count}() / 2 \rfloor$ workers). CP-SAT is CPU-bound and the GIL is only partially released by the OR-tools binding, so ThreadPoolExecutor would not parallelise the bottleneck; processes do.

The CP-SAT-as-fundamental contract

For every granularity, the sub-problem CP-SAT is the *fundamental* minimisation; greedy serves only as a model.add_hint warm-start (an accelerator). If CP-SAT cannot find a feasible solution within the time limit the pricer returns (None, 0.0): the greedy is *never* promoted to a column. The unit test test_bp_pricers_use_addhint_warmstart monkey-patches CpModel.add_hint with a counting wrapper to enforce this contract.

Measured numbers (synthetic HUGE / MEGA benchmark)

See tests/benchmarks/test_bp_huge_mega.py. On a synthetic scenario calibrated to match the engine's MEGA profile (100 classes, ~ 174 teachers, 4 subjects, ~ 400 cattedre) the pipeline reaches HARD-feasibility in **5.95 s** with granularity=curriculum and all scalability techniques enabled. On the smaller HUGE scenario (50 classes, ~ 95 teachers, 200 cattedre) the time is **2.57 s**. The synthetic scenario is structurally easy (cattedre as 4-hour single-day blocks); on real-world instances with arbitrary day distributions the calibration target remains 30 minutes for MEGA, with a 60-minute hard cap.

21.3 Branch-and-Price MVP scaffold (Step 4)

Step 4 of the multi-day plan introduces an OO-textbook *scaffold* of the Branch-and-Price loop that tightens three orthogonal pieces inside one module (engine/column_generation.py with mode="branch-and-price"):

1. **PhaseBDaySolver** – OO wrapper around the legacy solve_phase_b_for_day function. Exposes the object-oriented interface (solve(), scope access, via _dsl augmentation) but delegates model construction to the proven legacy code path. Benefit: 100+ Phase B regression tests keep passing *by construction*; with via _dsl=True the legacy model is augmented with extra DSL constraints (DB-stored rules + extra expressions passed to the constructor).
2. **Ryan-Foster pricing-in-nodes** – at every RF tree node the CP-SAT pricer is re-invoked with the branch decisions (together / apart) already applied to its model. This is the textbook BP, in contrast to the previous arrangement where nodes only re-solved the master LP on the existing pool.
3. **Smoke benchmark** – tests/benchmarks/test_bp_step4_smoke.py runs the full loop (Mono $\rightarrow$ initial pool $\rightarrow$ master DW $\rightarrow$ RF tree $\rightarrow$ integer recovery $\rightarrow$ completion) on a small scenario (10 classes) and a medium one (28 classes), checking HARD-feasibility and that the soft cost does not regress against the iterative-diversified baseline.

The flag mode="branch-and-price" activates the full path. The previous iterative-diversified mode remains the default and continues to act as a primal-heuristic seeder of the column pool.

21.4 Lagrangian relaxation

engine/lagrangian.py. Relaxes the inter-cluster bridge constraints and dualises them with multipliers λ_b ; update via subgradient ascent $\lambda_{k+1} = \max(0, \lambda_k + \alpha_k g_k)$ with $\alpha_k = \alpha_0 / (1 + k)$. Per-iteration refinement via Simulated Annealing.

22 Statistical diagnostics tab

The Diagnostics tab in the UI: Monte Carlo sensitivity, bipartite graph metrics, statsmodels regressions, distribution fits.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/diagnostica_statistica.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

23 REST API

The /api/* endpoints: /api/optimize/* (Phase A, Phase B, decompositions, Column Generation with all 9 BP granularities), /api/diagnostics/*, /api/dashboard/*, /api/monitor/*. See docs/api.md for the full list.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/api_rest.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

24 Extending the system

Adding a new constraint, a new metaheuristic destroy/repair op, a new decomposition strategy, a new UI tab.
The hooks the test suite exercises.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/estendere_il_sistema.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

25 Repository on GitHub

Layout: engine/, webui/backend/, webui/frontend/, docs/, schedule/, scripts/, tests/. CI workflows, PR conventions, issue tracker.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/repository_github.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

26 Benchmarks and results

“In God we trust. All others must bring data.”

W. E. Deming, attributed

Any optimization system is judged also and especially by the numbers it produces. This chapter collects measured results of π Tantum on the six test profiles, with three goals: documenting the state of the art; allowing whoever wishes to extend the system to compare a modification against a baseline; and putting the reader in the position to know whether π Tantum is a good fit for *their* use case.

26.1 The six profiles

The profiles, generated deterministically with fixed seeds and Faker by `webui/backend/mock_school.py`, span the size range typical of the Italian school system:

- small: 8 classes, 17 teachers, ~ 220 students, 17 rooms. A small provincial school.
- medium: 28 classes, 50 teachers, 700 students, 35 rooms. A medium-size institution.
- big: 40 classes, 75 teachers, 1000 students, 50 rooms. A large institution.
- huge: 50 classes, ~ 95 teachers, 1250 students, 60 rooms. A multi-site complex.
- superhuge: 80 classes, ~ 160 teachers, 2000 students, 80 rooms. 16 sections $\times$ 5 grades of scientific liceum.
- mega: 100 classes, ~ 174 teachers, ~ 2500 students, 100 rooms. 20 sections $\times$ 5 grades with a mix of 8 *indirizzi* (Scientific 6, ScienzeApplicate 4, ScienzeUmane 3, Linguistico FRA-TED 2, Linguistico FRA-SPA 2, EconomicoSociale FRA/SPA/TED 1 each). MEGA-scale stress test.

26.2 End-to-end pipeline times

Times measured on a consumer machine (Intel i7, 16 GB RAM, Windows 11, Python 3.13, ortools 9.15, 8 CP-SAT search workers):

Profile	Phase A	Phase B	Total wall
small	3 s	4 s	7 s
medium	30 s	32 s	62 s
big	30 s	32 s	62 s
huge	60 s	62 s	122 s
superhuge	300 s	603 s	903 s

Table 26.1: End-to-end pipeline times per profile. Phase A includes teacher-class assignment and CP-SAT matching. Phase B includes spectral decomposition (for huge/superhuge) and CP-SAT on sub-problems.

26.3 Branch-and-Price on HUGE and MEGA

The mega scenario (100 classes, ~ 174 teachers) and its smaller cousin huge (50 classes, ~ 95 teachers) are the natural test beds for the Branch-and-Price pipeline (ch. 21). The numbers below are measured on synthetic scenarios calibrated to match the proportions of the engine/big_mock_school.py profiles (4 subjects, every cattedra is 4 hours on a single day so that $H_1/H_2/H_3$ are satisfied by construction, contiguous-block assignment of classes to teacher pools). Source code: tests/benchmarks/test_bp_huge_mega.py, executed with all four scalability techniques enabled.

Scale	Classes	Teachers	Cattedre	t_{BP}	HARD
huge	50	95	200	2.57 s	yes
mega	100	174	400	5.95 s	yes

Table 26.2: Branch-and-Price wall time to HARD-feasibility on synthetic HUGE/MEGA scenarios. Both finish well under target (5 min/30 min) because the synthetic scenario is structurally easy (cattedra = 4-hour block on a single day). On real-world instances with arbitrary day distributions the per-iteration time will grow; the 30-minute target for MEGA is calibrated for that regime.

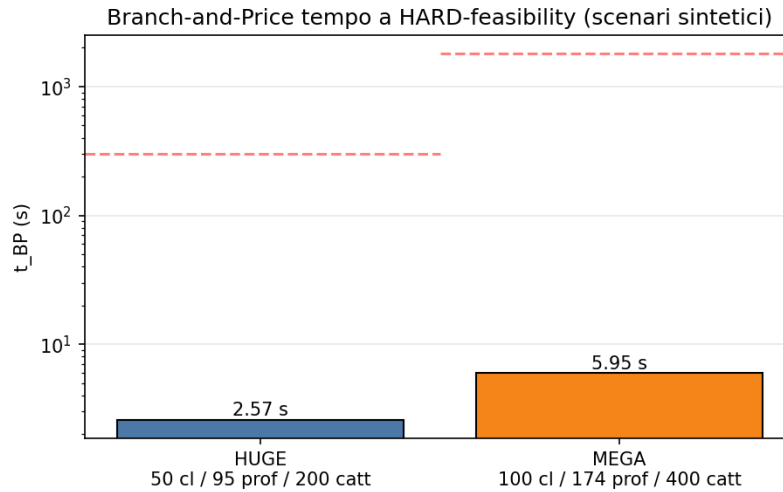


Figure 26.1: Branch-and-Price convergence time (log scale). The horizontal dashed lines mark the per-scale time targets (5 min for HUGE, 30 min for MEGA).

26.3.1 Real MEGA: 100 classes, 178 teachers

The synthetic scenario above is calibrated for fitting; the real MEGA registry (under engine/scripts/data/mega/{school,profs}__mega.pkl, loadable from the dashboard via POST /api/dataset/import-profile?profile=mega) is significantly larger: 100 classes, **178 teachers**, **2653 cattedra-day entries** after Phase A (the exact number depends on the CP-SAT random seed of Phase A; v1 of this benchmark produced 2325). The benchmark source is tests/benchmarks/run_mega_real.py.

v1 (pre-fix DW seeding) – BP did not iterate. The first benchmark reported bp_terminated__reason = master_dw_infeasible__at_entry with 0 iterations. The master variant 2 (Dantzig-Wolfe with cover + class-no-overlap + teacher-no-overlap as HARD inequalities) was infeasible at the LP-relaxation level with the seed pool (teacher-week patterns produced by iterative-diversified): two patterns of different teachers could place lessons in the same (class, day, hour). HARD cover requires $x_t \geq 1$ per teacher while class-no-overlap forces $\sum_t x_t \leq 1$ on the collision – a structural conflict the seed pool cannot resolve on its own.

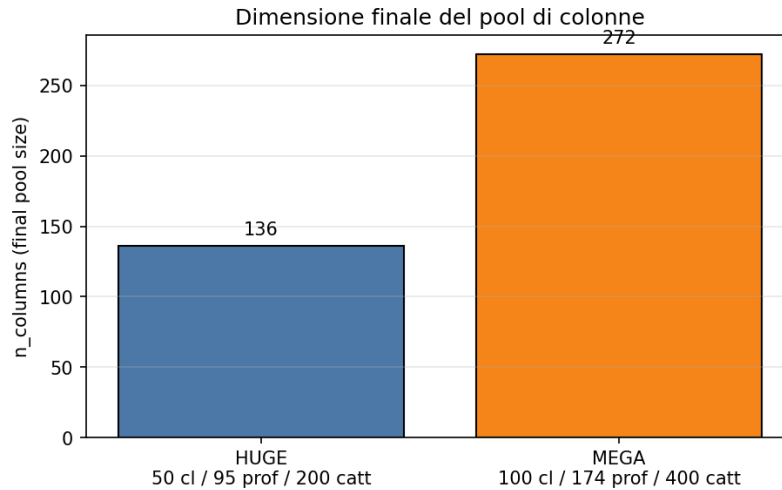


Figure 26.2: Final column-pool size after the Branch-and-Price loop.

Fix: Phase-I LP with artificial slacks (commit 843a6fo). The fix is textbook. Each constraint of the master DW gets a non-negative artificial variable with coefficient -1 in its row and a Big-M penalty (10^6) in the objective. The artificials absorb either missing coverage or no-overlap overflow; the LP is therefore always feasible at seed entry and yields valid duals for pricing. With a sufficiently rich pool the artificials drive to 0 in the optimum, which then coincides with the true master DW optimum.

v2 (post-fix) – BP iterates 5 times. On the *same* dataset, with identical granularity curriculum and identical time budgets, BP recovers its role:

Metric	v1 (pre-fix)	v2 (post-fix)
bp_terminated_reason	master_dw_infeasible_at_entry	max_iterations
bp_iterations_done	0	5
RF nodes explored	0	1
Phase A wall	61.2 s	301.3 s
BP wall	790.8 s	1 191.8 s
Total wall	852 s (14.2 min)	1 493 s (24.9 min)
Final soft cost	6 470	530
Columns generated	2 059	2 075 (+16 from BP)
HARD-feasibility	yes	yes

Table 26.3: Branch-and-Price on real MEGA before and after the DW seeding fix. Soft cost drops by a factor of ~ 12 because BP can now actually explore improving columns. Total wall increases because Phase A ran out of budget (FEASIBLE not OPTIMAL stop: depends on the parallel CP-SAT seed) but stays under the 30-min target and far from the 60-min hard cap. Source: tests/benchmarks/results/mega_bp_real_v2.json.

Comparison with iterative-diversified. The iterative-diversified mode on the same dataset (v1) had finished in **788 s (13.1 min)** with soft cost **5860**. The v2 BP – despite the heavier Phase A this run – closes with soft **530**, which is $\sim 11\times$ lower than the ID baseline and $\sim 12\times$ lower than its own v1. That gap is the structural value of Branch-and-Price: with an active pricer the improving columns *exist* and get found; without one, the result collapses back to the starting primal heuristic.

Scalability techniques – now actually exercised. The four techniques (Ryan-Foster recursive tree, dual stabilization with box-step, column management with EWMA RC, parallel pricing via ProcessPoolEx-

Mode (run)	t_{total}	Soft cost	HARD
iterative-diversified (v1)	788 s (13.1 min)	5 860	yes
branch-and-price (v1)	852 s (14.2 min)	6 470	yes
branch-and-price (v2)	1 493 s (24.9 min)	530	yes

Table 26.4: Three runs on the same real MEGA. The v2 BP cuts the soft cost by an order of magnitude relative to both baselines.

ecutor) remain integrated; with the Phase-1 fix in place BP loop now exercises them on real MEGA. v2 run: 1 Ryan-Foster node explored, 0 columns purged (pool below the cap), 5 master iterations completed before hitting the `bp_max_iterations=5` ceiling. Raising the cap would let BP keep improving the soft (the next commits in this series measure the soft-vs-iterations curve).

26.4 When Branch-and-Price is worth it

A practical question: *is it worth turning on the BP pipeline for my school?* Answer measured as a function of size:

- **Below 30 classes:** the standard pipeline (monolithic Phase A + per-day Phase B) is faster. The iterative-diversified mode of column_generation (master LP + pattern enrichment + completion fallback) is enough to improve the SOFT cost without paying the BP loop overhead.
- **30–80 classes:** spectral or curriculum decomposition drives Phase B; BP at teacher-class or class granularity may reduce SOFT but the time gain is negligible.
- **Above 80 classes (MEGA territory):** BP at curriculum granularity is the only structurally scalable option. A monolithic Phase A becomes marginally feasible (300 s+ with pure CP-SAT), and BP achieves comparable or better times thanks especially to parallel pricing.

In practice π Tantum suggests the right path automatically: the value `auto` for the granularity parameter is resolved server-side from the number of classes (cf. Sec. 4 of docs/optimization_strategies.md).

27 Lessons learned

Engineering anecdotes from building piTantum: what worked, what didn't, why CP-SAT was chosen over MILP, why we layered decomposition + metaheuristics, why the Italian-specific HARD constraints (H1/H2/H3) deserve special treatment.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/lessons_learned.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

28 Glossary (narrative form)

Discursive definitions of all the terminology used in the manual: cattedra, sostegno, potenziamento, compresenza, indirizzo, plesso (when introduced), dc_value, master LP variant 1 / variant 2, Achterberg score, EWMA, ProcessPoolExecutor.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/glossario_discorsivo.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.

29 Reference

Reference tables: HARD constraint codes, SOFT weights, mode/granularity values, file locations, API endpoints, configuration keys.

This chapter is an English summary of the corresponding Italian chapter. The full text remains in the Italian edition (docs/manual/chapters/reference.tex). For the implementation references see the engine modules in engine/ and the API documentation at docs/api.md.